Learning to Act: Methods of Reinforcement Learning

Alexandra Boog, Soros Qin
University of California, Santa Barbara

History

Reinforcement Learning (RL) emerged in the late 1970s from psychology, neuroscience, and control theory, focusing on hedonistic systems that maximize reward through trial and error. Today, RL powers real-world applications in gaming, web services, finance, and healthcare, with the goal of learning through interaction for better decision-making.

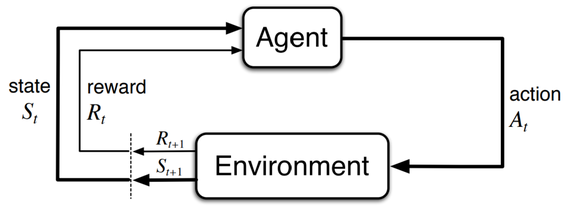


Figure 1: the action-agent interaction in a Markov Decision Process (MDP).

Background

Fundamentally, Reinforcement Learning uses mathematical methods to find the **optimal policy**, π^*, that an agent can follow to achieve a specific goal in an environment. This is done by having the agent and environment interact in a specific **action**, $a \in A$ from a **state**, $s \in S$, placing the agent in a new *state*, $s' \in S$. A policy π tells us which action to take in a given state, also known as a **state-action pair**. Each *state-action pair* gives us a different **return**, which is the total accumulated reward, r, that the agent seeks to maximize.

We assign a **value function** to each *state*, v_π, and *state-action pair*, q_π, that's dependent on a specific policy and we define one policy to be better than the other if it's expected return is greater. Using this logic, we can find the **optimal policy** by finding out which policy maximizes these *value functions*. As time progresses, the influence of future rewards diminishes. This is shown using a **discount factor**, $\gamma \in [0, 1]$, which reduces the contribution of rewards received further in the future.

Key Equations (Bellman optimality):

$$v^*(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v^*(s')\big],$$

$$q^*(s, a) = \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma \max_{a'} q^*(s', a')\big].$$

We can represent this by a backup diagrams, which show how we gather information backwards form future states to update our *optimal policy*.

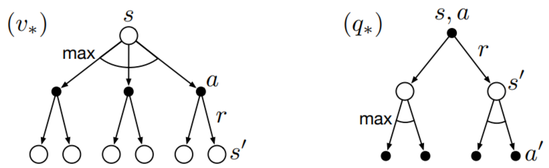


Figure 2: Backup Diagrams for v_* and q_*

Acknowledgements

We thank Jack Pfaffinger for his guidance as well as the UCSB Directed Reading Program for the opportunity to work on this project.

References

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Cambridge, Massachusetts; London, England: The MIT press, 2018.

Dynamic Programming (DP)

Definition. A family of algorithms for computing optimal value functions and policies when the full MDP model is known. It uses the known transition/reward model $p(s', r \mid s, a)$ to perform *expected updates* (policy evaluation, iteration, value iteration). DP performs **expected updates** using the Bellman equations to refine value estimates. We begin with **iterative policy evaluation**, where an arbitrary v_0 is updated under a given policy π by applying the Bellman equation as the **Key Update:**

$$v_{k+1}(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma v_k(s')\right].$$

As this process repeats $v_k \longrightarrow v_\pi$. **Policy improvement** chooses actions that maximize expected return, aka what actions maximize $q_\pi(s, a)$. This gives us a new, *greedy* policy.

$$\pi'(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma v_\pi(s')\right].$$

Alternating these processes is **policy iteration** and it yields increasingly better policies with repetition until we eventually converge to the optimal policy, π^*.

$$\pi_0 \xrightarrow{\mathsf{E}} v_{\pi_0} \xrightarrow{\mathsf{I}} \pi_1 \xrightarrow{\mathsf{E}} v_{\pi_1} \xrightarrow{\mathsf{I}} \pi_2 \xrightarrow{\mathsf{E}} \cdots \xrightarrow{\mathsf{I}} \pi_* \xrightarrow{\mathsf{E}} v_*$$

Monte Carlo (MC)

Definition. Methods for estimating value functions by averaging sample returns from complete episodes, without requiring any model. The return G_t from time step t is defined as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

The value function is updated using the **Key Update (constant-α first-visit MC):**

$$V(s) \leftarrow V(s) + \alpha\big(G_t - V(s)\big),$$

where $\alpha > 0$ is the learning rate. There are two main types of MC Methods: **first-visit** and **every-visit** which estimate $v_\pi(s)$ following the first visit or all visits to s respectively.

Off-Policy

MC methods estimate value functions based on complete episodes of experience. This allows us to evaluate a target policy π, while generating episodes using a different behavior policy b. To correct for the difference, we use **importance sampling**, which re-weights returns based on how likely we are to choose an action-state pair under π versus b.

$$\text{importance sampling ratio: } \rho_t = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}$$

The basic form, *ordinary importance sampling*, scales each return by ρ_t and averages the results. Although unbiased, this method can have high variance if b differs greatly from π. Although biased, *weighted importance sampling* solves this and reduces variance by normalizing the weights.

$$\text{ordinary: } V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_t\, G_t}{|\mathcal{T}(s)|} \qquad \text{weighted: } V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_t\, G_t}{\sum_{t \in \mathcal{T}(s)} \rho_t}$$

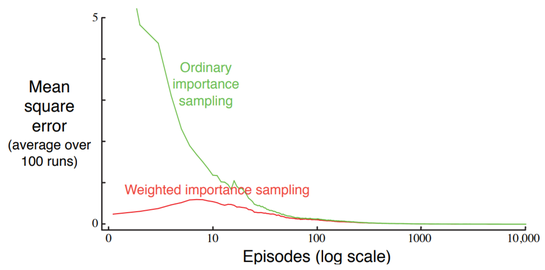
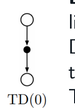


Figure 3: Weighted importance sampling produces lower error estimates of the value of a single blackjack state (normal rules) from off-policy episodes.

Temporal Difference (TD)



Definition. A hybrid of MC and DP that learns directly from experience like MC and **bootstraps**—updating estimates using other estimates— like DP without requiring a model. While MC waits until the end of an episode to update, TD updates after a specified number of steps. The special case, TD(0), updates after each step, but the more general form, TD(n) updates after n steps.

Key Update (TD(n)):

$$V_{t+n}(S_t) = V_{t+n-1}(S_t) + \alpha\left[G_{t:t+n} - V_{t+n-1}(S_t)\right], \quad 0 \le t < T$$

Note that the quantity in brackets is the **TD error**, δ_t, which is the difference between successive estimates. This method of updating is incremental which allows for quick updates.

n-step SARSA (on-policy)

We estimate the action-value function $q_\pi(s, a)$ under the current policy π, using the same n-step update structure we described above for state values.

$$Q_{t+n}(S_t, A_t) = Q_{t+n-1}(S_t, A_t) + \alpha\left[G_{t:t+n} - Q_{t+n-1}(S_t, A_t)\right], \quad 0 \le t < T$$

Q-Learning (off-policy)

We directly approximate q_*, independent of the policy being followed, by using the maximum estimated value at the next state (from TD(0) specifically).

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_a Q(A_{t+1}, a) - Q(S_t, A_t)\right].$$

n-step Expected SARSA (on-policy)

We average over the expected value over the distribution of future *state-action pairs* instead of the max. Thus assuming $t + n < T$ and G_t is the return function,

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \sum_a \pi(a \mid S_{t+n}) Q(S_{t+n}, a)$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[G_{t:t+n} - Q(S_t, A_t)\right]$$

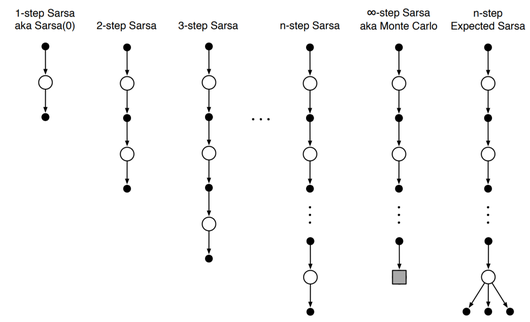


Figure 4: The backup diagrams for the spectrum of n-step methods for state–action values.

Trade-Offs

Dynamic Programming (DP)
Pros: Converges to exact solution, uses full model.
Cons: Requires known dynamics, computationally expensive (not scalable).

Monte Carlo (MC)
Pros: Model-free, easy to implement.
Cons: High variance, needs complete episodes (can be slow), no bootstrapping.

Temporal Difference (TD)
Pros: Model-free, bootstraps, lower variance (and faster) than MC, online and incremental.
Cons: Biased updates, analysis more complex than MC.